

System Architecture Document

Project: CareerTailor (Production V1.1)

Date: February 2026

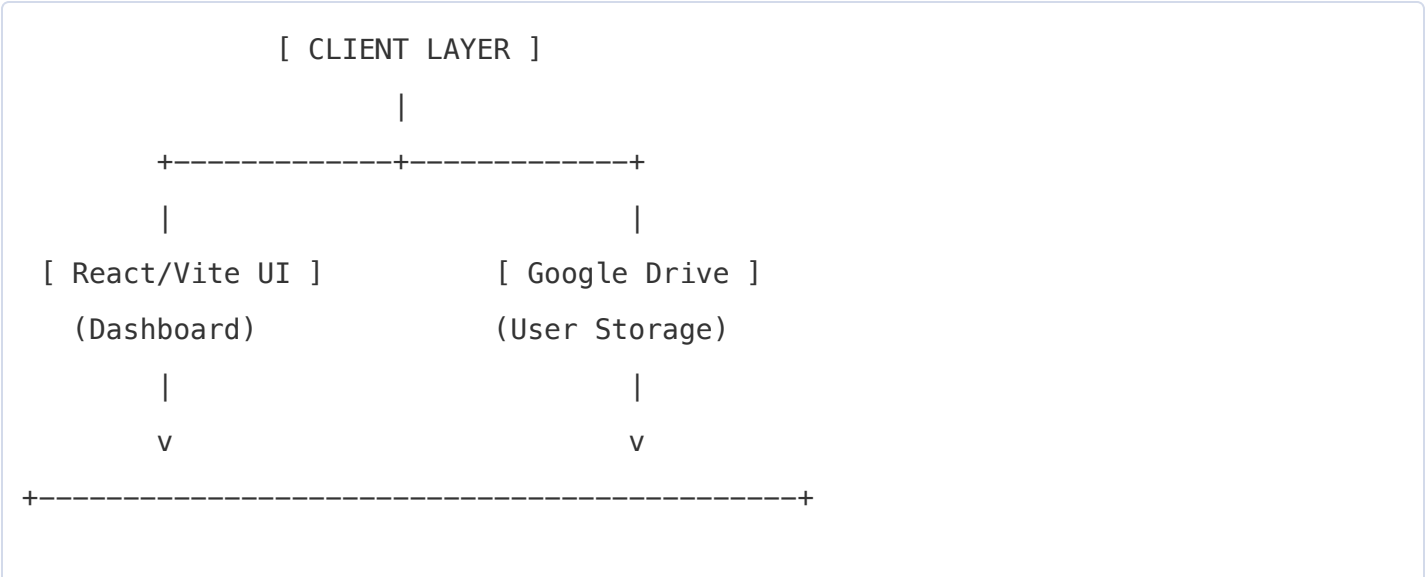
Status: Approved for Development

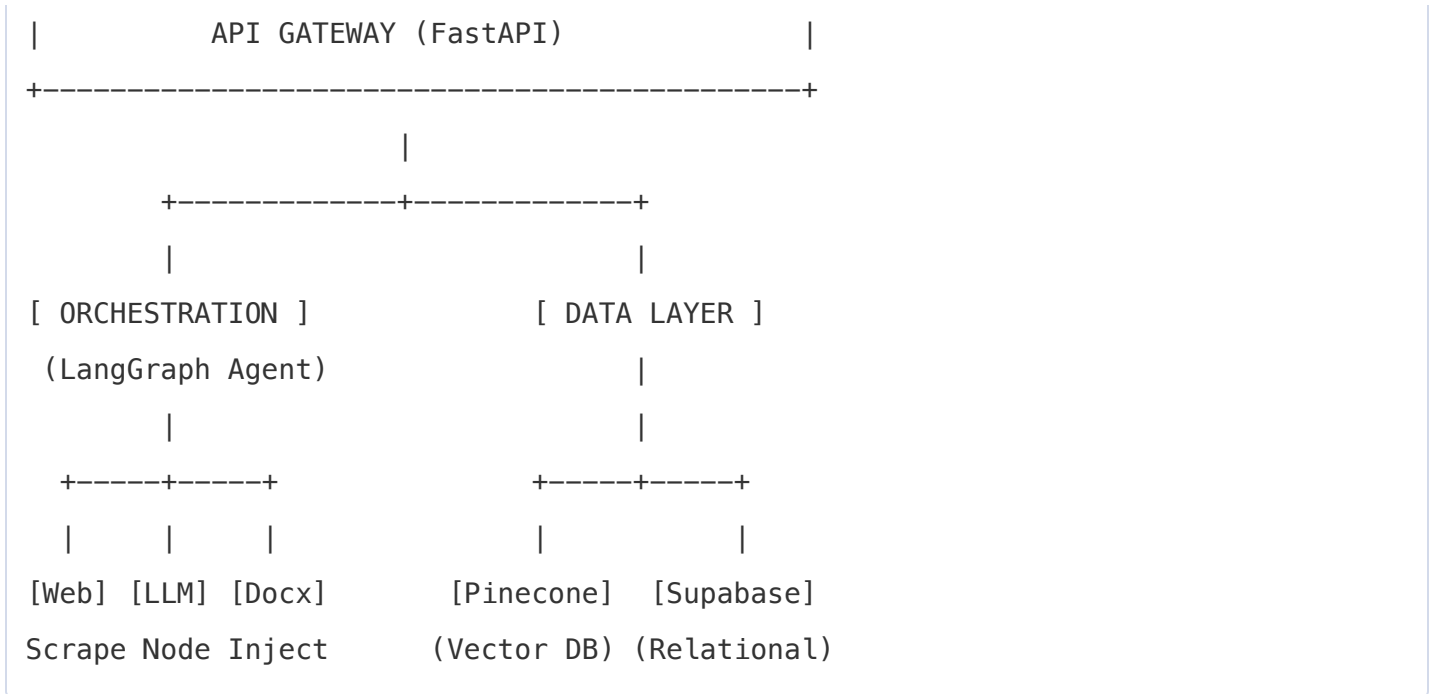
Version History & Changelog

Version	Date	Author	Changes Made	Reason for Change
1.0	Jan 2026	System Architect	Initial Architecture. Defined Batch Triage, LangGraph Orchestration, and Postgres “Skill Bank”.	Baseline system design.
1.1	Feb 2026	System Architect	1. RAG Integration: Replaced Postgres <code>user_skill_bank</code> with Pinecone Vector DB . 2. Added Ingestion Phase: Master Resumes are now chunked and embedded before triage.	To eliminate LLM hallucination. Standard relational tables only store boolean keyword matches (e.g., “Python = True”). A Vector DB stores the <i>semantic meaning</i> of past experiences, allowing the LLM to retrieve exact historical context when rewriting resume bullet points. Aligns with industry-standard AI Engineering practices.

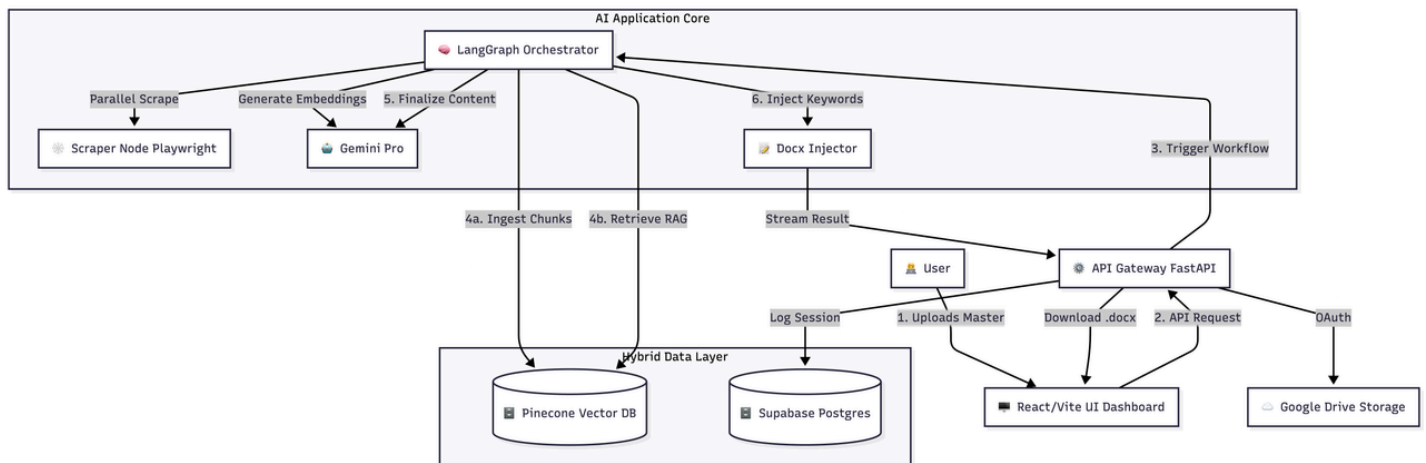
1. High-Level Architecture Diagram (V1.1)

The following flow describes the new ‘RAG Ingestion’, ‘Batch Triage’, and ‘Resume Factory’ logic:





2. Component Interactions & Data Flow



2.0 Phase Pre-A: RAG Ingestion (New)

1. User uploads their “Master Resume” via the Frontend.
2. API Gateway extracts text and splits it into semantic chunks (using `RecursiveCharacterTextSplitter`).
3. Chunks are passed through an Embedding Model (Gemini `text-embedding-004`) to convert text to high-dimensional vectors.
4. Vectors and metadata are upserted to the **Pinecone Vector Database**.

2.1 Phase A: Batch Triage (The “Scan”)

- 1. User submits 3-5 Job URLs via Frontend.
- 2. API Gateway creates a ‘Session’ in **Supabase (Postgres)**.
- 3. LangGraph spawns parallel ‘Scraper Nodes’ (using DOMContentLoaded & specific selectors).
- 4. ‘Analyst Node’ computes a Feasibility Score (0-100%) using Gemini.
- 5. Frontend receives a ‘Traffic Light’ status (Green/Yellow/Red).

2.2 Phase B: Unified Interview (The “Semantic Bridge”)

- 1. System calculates the UNION of missing skills across selected jobs.
- 2. Agent embeds the missing skills and performs a **Cosine Similarity Search** in **Pinecone**.
 - **IF Semantic Match Found:** Auto-verifies the skill using the retrieved contextual experience.
 - **IF Missing:** Adds to the ‘Interview Queue’.
- 3. User answers questions once via Chat UI.
- 4. User’s new answers are instantly embedded and saved to Pinecone for future use.

2.3 Phase C: Generation (The “Factory”)

- 1. ‘Injector Node’ reads the Master Resume (Local bytes or GDrive ID).
- 2. Node creates a copy: **Resume-[Company]-[Role].docx** .
- 3. Node uses retrieved context from Pinecone to intelligently inject keywords into specific paragraph runs (preserving formatting).
- 4. File is streamed to user (Local) or saved to Drive (Cloud).

3. Storage Strategy (Hybrid RAG Approach)

To maintain absolute user privacy and high AI performance, data is strictly separated:

Data Type	Storage Location	Retention Policy	Purpose
Session State & History	Supabase (Postgres)	Persisted	Audit logs, UI history sidebar, app state.

Data Type	Storage Location	Retention Policy	Purpose
Semantic Memory	Pinecone (Vector DB)	Persisted	RAG Embeddings, resume chunks, context retrieval.
Scraped Job Text	Redis / Cache Table	24 Hours (TTL)	Temporary storage for parallel scraping.
Resume Files (Binary)	RAM / Google Drive	Ephemeral / User Owned	Complete privacy; we do not store the master .docx file.